

Unity MIDI Plugin — ドキュメント目次

このドキュメントでは、Assets/MIDI の内容、ランタイム API の構造、および Unity で MIDI 1.0、MIDI 2.0 (UMP)、MPE、および関連するトランスポートを使用する方法について説明します。

NotebookLM

このマニュアルを登録したAIを作成してみました。不明点があれば、まずはこちらに問い合わせてみましょう。(Google アカウントが必要です)

<https://notebooklm.google.com/notebook/10ea85d5-92d8-4225-bd8b-b24e4e1bf747>

言語

- [English](#)
- [中文\(简体\)](#)

目次

はじめに

- [スタートガイド \(インストール、初期化、送受信\)](#)
- [ビルド後の処理 \(PostProcessing\) とスクリプト定義シンボル](#)
- [プラットフォームと制限事項](#)

コア API

- [MIDI 1.0 \(MidiManager\)](#)
- [MIDI 2.0 / UMP \(Midi2Manager\)](#)
- [MPE \(MIDI Polyphonic Expression\)](#)
- [SMF / シーケンシング \(jp.kshoji.midisystem\)](#)

ユーティリティとエディタツール

- [ユーティリティ \(MidiNoteUtility / MidiMessageBuilder / PitchBend / Timing\)](#)
- [エディタツール \(Monitor / Virtual Controller / SMF Preview / Project Settings\)](#)

ゲームプレイ向けコンポーネント

- [ゲームプレイ向けコンポーネント \(InputMap / NoteTracker / Filter\)](#)
- [SMF ツール \(SmfPlayer / MidiRecorder / TempoMapExtractor\)](#)

Unity エコシステム統合

- [Unity 統合 \(Timeline / Animator / Visual Scripting / Input System / Scriptable Audio / Chunity\)](#)

ジャンルキット

- [ジャンルキット \(ネットワーク / Clock / 和音・スケール / Input System / Foundation\)](#)

トランスポートと統合

- [トランスポートとプラットフォーム](#)
- [アプリ間 MIDI — クロスプラットフォームの注意点 \(Android, iOS/macOS, Linux\)](#)
- [Maestro / MPTK 統合 \(仮想デバイス + アダプター\)](#)

プロジェクト構成の注意点

- [仮想デバイスとイベント注入](#)
- [MIDI-CI / 機能ネゴシエーション](#)
- [エディタとライフサイクルに関する注意点](#)
- [組み込み済みサードパーティモジュール](#)

サンプルとリファレンス

- [サンプル](#)
- [動作確認済みデバイス](#)
- [連絡先 / サポート](#)
- [更新履歴](#)

将来の機能（未実装）

以下は現行リリースには含まれていません。ロードマップとして参照してください。

別ドキュメントで既に提供済み（欠落扱いしないでください）：[ピッチベンド正規化 / 14-bit 分解](#)（[Utilities](#) の [PitchBendUtility](#)）、[Channel/Device フィルタ](#)（[Gameplay](#)）、[ランタイムレイテンシ校正](#)（Foundation の [MidiLatencyCalibrator](#) / [Genre Kits](#)）、[SMF Preview](#) からの一方向 SMF → JSON エクスポート。

カテゴリ	機能	概要
エディタ	デバイスブラウザ	<code>Window > MIDI > Device Browser</code> — 接続デバイス一覧、Vendor/Product ID、テスト送信
エディタ	Scene View デバッグオーバーレイ	Play 中にシーン上へ押下ノート・直近メッセージを表示
エディタ	レイテンシ計測（RTT ツール）	エディタ上の送信→受信往復 UI（BLE / RTP-MIDI 評価用。Foundation のタブ校正とは別）
ゲームプレイ	MPE 高レベル API	<code>MpeInputHandler</code> — Per-note 表現を統合した <code>UnityEvent</code> （低レベルは MPE ）
ゲームプレイ	MIDI イベントルーティング	複数 <code>MidiInputRouter</code> 間の優先度・排他制御
ユーティリティ	14-bit CC	<code>MidiCc14BitUtility</code> — CC の MSB/LSB ペア組み立て・分解
ユーティリティ	ピッチバンド → セミトーン	14-bit ピッチバンドを設定可能なセミトーン範囲へ変換（正規化 / Split は実装済み）
ユーティリティ	UMP パーサーヘルパー	<code>UmpParser</code> — UMP ワード列の型付き分解
ユーティリティ	SysEx ビルダー / パーサー	<code>MidiMessageBuilder.SystemExclusive</code> / 送信 API を超える高レベル組み立て・解析
ユーティリティ	SMF ↔ JSON（ランタイム双方向）	ランタイムでの双方向変換（エディタ Preview の一方向 JSON エクスポートは実装済み）
Chunity	MIDI 2.0 / UMP マッピング	32-bit velocity / Per-note controller → Chuck グローバル（現行は MIDI 1.0 経路）
Chunity	InstanceTarget API 完全化	<code>SetString</code> / <code>ListenForChuckEventOnce</code> / <code>Get*Array</code> / <code>RunFile</code> 引数など公式 API 面の薄いラップ
Chunity	Syncer / Poller	<code>Chuck*Syncer</code> / <code>EventListener</code> ラップによる Chuck → Unity 読み戻し
Chunity	連想配列・*_AT・VM 制御	名前付きパラメータ、オーディオスレッド書き込み境界、 <code>SetRunning</code> / 停止 Event 規約
Chunity	UGen プローブ / Host Time Advancer	波形可視化、Unity 駆動の Chuck 時間（Clock 同期の補完）
Chunity Generator	最適化	ネイティブポインタ API、Burst、Scriptable Effect / Root Output（必要時）

ディレクトリ構造 (Assets/MIDI)

- **Plugins/**

プラットフォームごとのネイティブ（および WebGL JS）プラグイン
(Android/iOS/macOS/Linux/WSA/WebGL)。

- **Scripts/**

メインの C# ランタイム:

- `MidiManager.cs` (MIDI 1.0)
- `Midi2Manager.cs` (MIDI 2.0 / UMP)
- プラットフォームプラグイン: `MidiPlugin.*.cs` , `Midi2Plugin.*.cs`
- イベントハンドリングインターフェース: `IMidi*EventHandler` , `IMidi2*EventHandler`
- MPE: `MpeManager.cs` , `IMpeEventHandler.cs`
- MIDI-CL: `MidiCapabilityNegotiator.cs`
- 仮想デバイス: `MidiManager.VirtualDevices.cs`

- **Scripts/Utilities/**

ドメイン別ランタイムヘルパー（名前空間 `jp.kshoji.unity.midi.util`）:

- **Messaging/** — `MidiOutgoingMessage` , `MidiMessageBuilder` , `PitchBendUtility` , `MidiControlSmoothing` , `AutomationInterpolation` , ...
- **MusicTheory/** — `MidiNoteUtility`
- **Smf/** — `TempoMapExtractor` , `TempoMap` , `MeasureTimeUtility` , `TupletUtility` , `SwingUtility` , `MidiMetaMessageFactory` , ...
- **Monitor/** — `MidiMonitorLogData` , `MidiMonitorFormatHelper`

- **Scripts/Editor/**

エディタ拡張:

- `PostProcessBuild.cs` (ビルド後処理)
- `MidiMonitorWindow.cs` 等 (MIDI モニター)
- `VirtualMidiControllerWindow.cs` (仮想 MIDI コントローラー)
- `SmfPreviewWindow.cs` (SMF プレビュー / インポート)
- `MidiProjectSettingsProvider.cs` (Project Settings)
- `UI/PianoKeyboardElement.cs` (`MidiKeyboardLogic` を含む)

- **Resources/MidiProjectSettings.asset**

グローバル MIDI 設定（初回自動生成）

- **Scripts/Gameplay/**

ゲームプレイ向けコンポーネント:

- `MidiInputMap.cs` / `MidiInputRouter.cs` (MIDI → UnityEvent)
- `MidiNoteTracker.cs` (押下ノート状態管理)
- `MidiFilterBase.cs` / `MidiChannelFilter.cs` / `MidiDeviceFilter.cs` (イベントフィルタ)
- `SmfPlayer.cs` / `MidiRecorder.cs` / `MidiRecordingSession.cs` (SMF 再生・記録)
- `MidiClockSync.cs` / `MidiClockOutput.cs` / `MidiChordDetector.cs` (Clock 同期・和音判定)
- `MidiCcProcessorBase.cs` / `MidiCcSmoother.cs` / `MidiCcButton.cs` (CC 処理)

- Theory/ — ChordRecognition , MidiScaleUtility / ScaleUtility
- Scripts/Integrations/Animator/

Animator 統合 (MidiAnimatorDriver , MidiBlendTreeDriver)
- Scripts/Integrations/Timeline/

Timeline 統合 (MidiPlaybackTrack , MidiRecordTrack , マーカー)
- Scripts/Integrations/VisualScripting/

Visual Scripting ノード (MidiVisualScriptingBridge , イベント / アクションユニット)
- Scripts/Integrations/Networking/

ネットワーク MIDI 同期 (MidiNetworkHub / MidiNetworkClient 、オプション Mirror / Netcode / WSNet2 ブリッジ、 FEATURE_MIDI_NETWORK + ブリッジ用シンボル)
- Scripts/Integrations/InputSystem/

Input System ブリッジ (MidiInputSystemBridge , MidiSyntheticDevice 等、
FEATURE_INPUT_SYSTEM)
- Scripts/Integrations/ScriptableAudio/

Scriptable Audio Pipeline 統合 (FEATURE_SCRIPTABLE_AUDIO 、Unity 6000.3+)
- Scripts/Integrations/Chunitty/

Chunitty 連携 (FEATURE_CHUNITY 、ランタイム非同梱) と Scriptable Generator
(FEATURE_CHUNITY_SCRIPTABLE_AUDIO)
- Scripts/Foundation/

Foundation (MidiLatencyCalibrator , MidiDeviceSelection , MidiOutputRoutingPreset ,
MidiFoundationUiController 等)
- UI/Foundation/

Foundation UI 用 UXML / USS
- Settings/

共有 Input System アセット (例: MidiController.inputactions)
- Scripts/MidiSequenceAsset.cs

SMF ScriptableObject (Create > MIDI > Sequence Asset)
- Scripts/UmpSequenceAsset.cs

UMP .midi2 ScriptableObject
- Scripts/MidiProjectSettings.cs

グローバル MIDI 設定 ScriptableObject
- Scripts/midisystem/

標準 MIDI ファイル (SMF) リーダー/ライター + シーケンシングモデル (Sequence , Track , メッセージ)。
- Scripts/UmpSequencer/

UMP シーケンスユーティリティ (クリップ/コンテナの読み書き、シーケンサー、SMF↔UMP コンバータ)。
- Scripts/RTP-MIDI-for-.NET/

RTP-MIDI 実装 (組み込みモジュールとそのドキュメント)。
- Scripts/UdpMidi2Discovery/

UDP MIDI 2.0 ワークフロー用のディスカバリ依存関係。
- Samples/

サンプルシーンとスクリプト。

- `Samples/Integrations/`
Unity エコシステム統合サンプル（Animator / Timeline / Visual Scripting / Networking / Input System / Chunity / Scriptable Audio）。
- `Samples/Gameplay/`
ゲームプレイサンプル（Clock 同期 / 和音・スケール判定）。
- `Samples/Foundation/`
Foundation サンプル（デバイス選択 / レイテンシ / UI シェル）。

概念と用語

- **Deviceld**: MIDI エンドポイントを参照するために API 全体で使用される文字列識別子。
- **Group**: MIDI 2.0 グループインデックス (0–15)。一貫性のために MIDI 1.0 API でも使用されます。
- **Channel**: MIDI チャンネル (0–15)。一部の API は 0 オリジンのチャンネル番号を使用することに注意してください。
- **UMP (Universal MIDI Packet)**: このプロジェクトでは `uint[]` ワードとして表現される MIDI 2.0 パケット形式。

クイックスタート・チェックリスト

1. 必要な機能セットを決定します:
 - MIDI 1.0 イベントと送信: `MidiManager`
 - MIDI 2.0 / UMP パースと送信: `Midi2Manager`
 - Android アプリ間 MIDI: [アプリ間 MIDI](#) を参照
 - MPE 管理: `MpeManager` (`MidiManager` 上で動作)
 - Inspector から MIDI → UnityEvent: [ゲームプレイ向けコンポーネント](#) の `MidiInputRouter`
2. 1 つ以上のイベントハンドラインターフェースを実装します。
3. ハンドラオブジェクトをマネージャーに登録します。
4. マネージャーを初期化します（またはシーン内に存在することを確認します）。
5. `Assets/MIDI/Samples/Scenes` にあるサンプルシーンを使用してテストします。
6. 開発中は `Window > MIDI > Monitor` で入出力メッセージを確認します（[エディタツール](#)）。